# Containerized Insurance Document Verification Portal

## Master DevOps Project Technical Report & Portfolio (Academic Year 2026-27)

**Candidate: Sourabh Shelke | Roll Number: 23102C0029**
**Class: BE CMPN (Computer Engineering) | Institute: Vidyalankar Institute of Technology**

## 1. Executive Summary

The Containerized Insurance Document Verification Portal is a production-grade enterprise microservice designed to automate the intake, validation, heuristic risk assessment, and fraud detection of insurance claim documents. Operating on a modern cloud-native stack (Java 21, Spring Boot 3.2, Maven, Selenium, Jenkins, Docker, Ansible), the system eliminates manual verification delays and prevents fraudulent payouts.

## 2. Technology Stack & DevOps Toolchain

| Layer | Tool / Framework | Purpose & Role in Project |
|---|---|---|
| Backend | Java 21 / Spring Boot 3.2 | Core business logic, REST controllers, JPA data access |
| Database | Embedded H2 Database | In-memory relational persistence with web console |
| Frontend | HTML5 / CSS3 / Vanilla JS | Responsive portal with live KPI analytics & audit modals |
| Build Tool | Apache Maven 3.9 | Dependency management, bytecode compilation, fat JAR assembly |
| Version Control | Git & GitHub | GitFlow branching, PR reviews, conflict resolution, tagging v1.0 |
| CI Server | Jenkins Automation Server | Freestyle CI job, SCM polling, declarative Jenkinsfile pipeline |
| Automated Testing | Selenium WebDriver & JUnit | 6 Critical user journey tests acting as CI quality gate |
| Containers | Docker & Alpine JRE | Lightweight container image, port 8085 binding, lifecycle testing |
| Config Management | Ansible Playbooks | Automated target node provisioning and idempotent execution |

## 3. Summary of 15 Completed DevOps Tasks

| Task ID | Milestone Summary | Verified Deliverable |
|---|---|---|
| Task 01 | Problem definition, stakeholders, scope freeze | Problem_Statement.md, Scope.md, Stakeholders.md |
| Task 02 | User stories, backlog, sprint plan, DoD, diagram | User_Stories.md, Backlog.md, Sprint_Plan.md, Diagram.md |
| Task 03 | SRS summary, architecture, REST API list, setup | srs_summary.md, architecture_diagram.md, API_List.md |
| Task 04 | Git repo init, README, .gitignore, branch policy | branch_policy.md, url.md, git_init_evidence.md |
| Task 05 | Feature branch (doc intake), PR, review, merge | feature_branch_workflow.md, pr_review_evidence.md |
| Task 06 | MVP completion, merge conflict resolution, v1.0 | merge_conflict_resolution.md, tag_evidence.md (v1.0) |
| Task 07 | Jenkins installation, Freestyle CI job, artifacts | jenkins_ci_guide.md, build_console_log.txt |
| Task 08 | Declarative Jenkinsfile pipeline with parameters | pipeline_doc.md, pipeline_execution_log.txt |
| Task 09 | Selenium test suite with 6 CUJs & screenshot tool | test_plan.md, test_execution_report.md |
| Task 10 | Jenkins continuous testing gate, defect fix rerun | quality_gate_docs.md, defect_and_fix_evidence.md |
| Task 11 | Docker container lifecycle & Alpine JRE image | docker_commands.md, container_lifecycle_log.txt |
| Task 12 | Jenkins-Docker CD commit-to-container pipeline | docker_cd_pipeline.md, cd_pipeline_log.txt |
| Task 13 | Ansible inventory & playbook for server config | config_spec.md, inventory.ini, setup.yml |
| Task 14 | Automated provisioning, idempotency, rollback demo | idempotency_validation.md, rollback_demo.md |
| Task 15 | Technical report, viva presentation guide, master doc | final_technical_report.md, viva_presentation_guide.md |

## 4. Key Results & Viva Preparedness

- **Processing Speed:** Automated intake and fraud assessment in under 200 ms.
- **Quality Gate:** 100% test pass rate across all 6 critical user journeys; defective code automatically halted.
- **Continuous Delivery:** Full commit-to-container pipeline executes in under 3 minutes.
- **Idempotent Provisioning:** Ansible playbook produces changed=0 on subsequent executions.